



와주개

믿고 말기는
우리동네 펫시터 매칭

PRESENTER.

이규화

DATE.

2026.07.08

TEAM.

멍발자들

FINAL PROJECT REPORT



INDEX

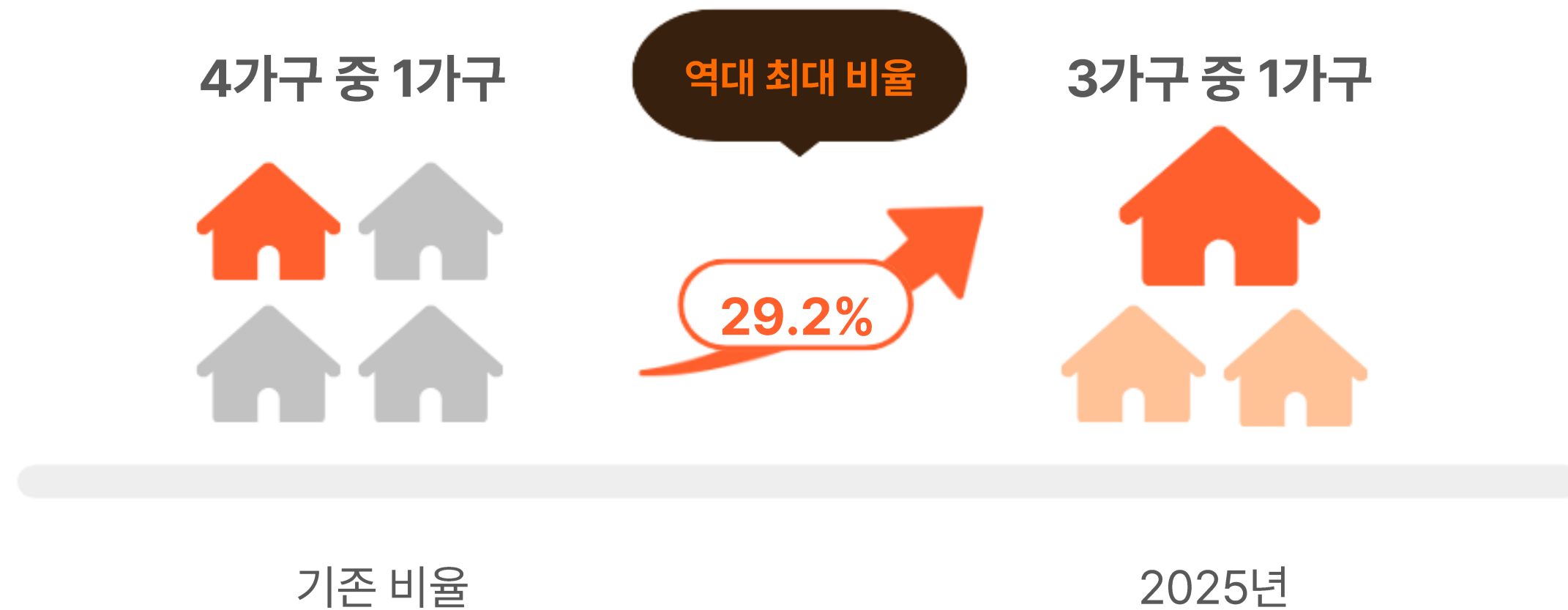
- Section. 1 프로젝트 개요
- Section. 2 주요 기능 시연
- Section. 3 Before / After — 트러블슈팅
- Section. 4 프로젝트 진행 방식
- Section. 5 팀원 소개 및 소감

01

프로젝트 개요

커지는 펫케어 시장, 그러나 아직 못 채운 신뢰의 공백

[2025년 국내 반려동물 양육가구 비율]



- ✓ 전국 17개 시·도의 3,000가구를 직접 조사한 결과, 3,000가구 중 867가구가 반려동물을 양육
- ✓ 국내 반려동물 시장 약 8.5조원 → 2032년 21조원 전망

[1인·맞벌이 가구 증가 → 산책·방문돌봄 수요↑]

- ✓ 1인 가구 증가에 따른
혼자 사는 가구가 늘어나며
자신을 대신한 산책·방문돌봄 가능 펫시터 수요 확대
- ✓ 반복되는 위탁 사고
휴가·출장 중 맡긴 반려동물의
질병·상해·사망 사고 발생
- ✓ 보호자의 불안 증가
"정말 믿고 맡겨도 될까?"
돌봄 선택 과정의 신뢰 문제 발생

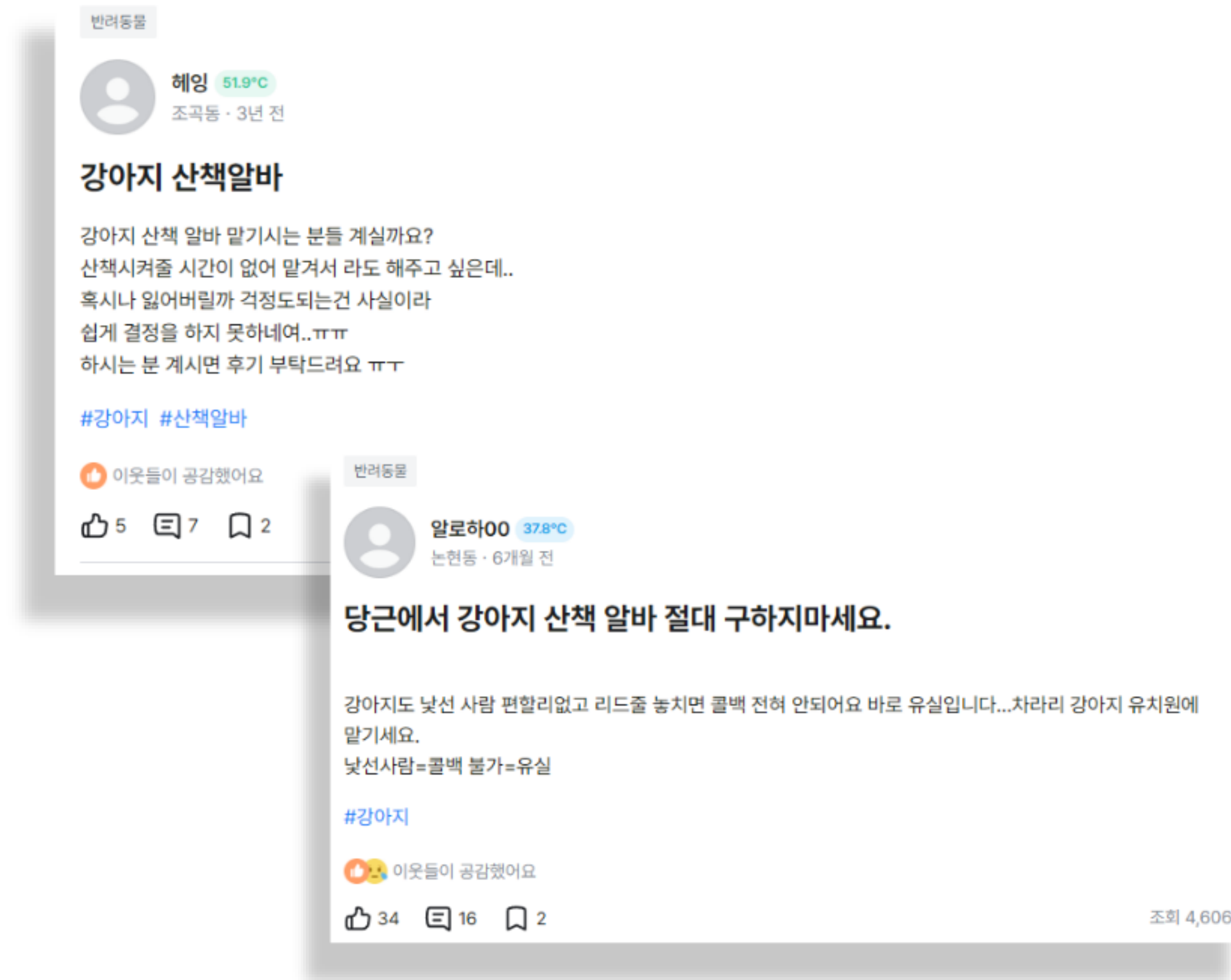
믿을 만한 펫시터, 어디서 찾아야 할까

01 문제

지역 기반으로 검증된 펫시터를 찾을 방법이 마땅치 않아,
지인 추천이나 오픈채팅에 의존하고 후기·신뢰 정보가 파편화

02 타겟

보호자 — 조건에 맞는 인근 펫시터를 지도에서 찾아 예약
펫시터 — 프로필을 등록해 돌봄 서비스를 제공하고 수익 관리



보호자와 펫시터, 각자의 여정을 하나의 플랫폼에서

봐주개 - lookpup

보호자와 펫시터를 연결하는
지역 기반 반려동물 돌봄 중계 웹 서비스

보호자는 주변 펫시터의 프로필, 후기, 제공 서비스, 가격을 비교하여 원하는 돌봄 서비스를 선택할 수 있습니다.

또한 산책, 방문 돌봄, 위탁 돌봄, 픽업 등 필요한 요청을 구인글로 등록하고 펫시터는 조건에 맞는 돌봄 업무에 지원할 수 있습니다.

봐주개는 예약, 결제, 후기, 신고, 지역 기반 검색 기능을 통해 보호자에게는 안심할 수 있는 돌봄 서비스를, 펫시터에게는 다양한 수익 창출 기회를 제공하는 것을 목표로 합니다.

#펫시터찾기

#돌봄요청

#돌봄예약

#채팅



봐주개

기술 스택

Framework	- Next.js 16 - React 19 - TypeScript	프레임워크
Styling	- Tailwind CSS - Radix UI - shadcn - Recharts	스타일링 · 헤드리스 컴포넌트
State/Data	- TanStack Query - Zustand - React Hook Form - Zod	서버 상태 · 클라이언트 전역 상태 폼 상태 · 스키마 유효성 검증
Backend/DB	- Supabase	Auth · DB · Realtime (Broadcast / postgres_changes)
결제	- tosspayments API	결제 연동
기타	- PortOne - Kakao Map - Cloundinary - Vercel	본인인증 · 지도·주소 · 이미지 업로드 · 배포

ERD-1

Table users		
Name	Type	Constraints
id	uuid	Primary
email	text	Nullable
provider	text	
full_name	text	Nullable
birthdate	date	Nullable
phone_number	text	Nullable
gender	text	Nullable
profile_image	text	Nullable
role	text	
deleted_at	timestampz	Nullable
created_at	timestampz	Nullable
updated_at	timestampz	Nullable
is_verified	bool	
location_consent	bool	
delete_reason	text	Nullable
latitude	numeric	Nullable
longitude	numeric	Nullable
address	text	Nullable
display_area	text	Nullable
suspended_until	timestampz	Nullable

Table pets		
Name	Type	Constraints
id	uuid	Primary
owner_id	uuid	
name	text	
animal_type	text	
breed	text	Nullable
age	int4	Nullable
gender	text	
weight	numeric	Nullable
image_url	text	Nullable
caution	text	Nullable
deleted_at	timestampz	Nullable
created_at	timestampz	Nullable
updated_at	timestampz	Nullable

Table sitters		
Name	Type	Constraints
id	uuid	Primary
user_id	uuid	Unique
title	text	Nullable
introduction	text	Nullable
career	text	Nullable
available_area	text	Nullable
latitude	numeric	Nullable
longitude	numeric	Nullable
base_price	int4	Nullable
status	text	
rating	numeric	Nullable
created_at	timestampz	Nullable
updated_at	timestampz	Nullable
request_type	_text	
certificate_urls	_text	
available_animals	_text	
activity_photo_urls	_text	
display_area	text	Nullable

ERD-2

Table services

Name	Type	Constraints
id	uuid	Primary
sitter_id	uuid	
title	text	
description	text	Nullable
service_type	text	
animal_type	text	Nullable
price	int4	
location	text	Nullable
latitude	numeric	Nullable
longitude	numeric	Nullable
image_url	text	Nullable
is_active	bool	
created_at	timestampz	Nullable
updated_at	timestampz	Nullable
deleted_at	timestampz	Nullable

Table requests

CHECK: start_datetime < end_datetime

Name	Type	Constraints
id	uuid	Primary
owner_id	uuid	
pet_id	uuid	Nullable
title	text	
content	text	Nullable
request_type	text	Nullable
start_datetime	timestampz	Nullable
end_datetime	timestampz	Nullable
budget	int4	Nullable
location	text	Nullable
latitude	numeric	Nullable
longitude	numeric	Nullable
status	text	
created_at	timestampz	Nullable
updated_at	timestampz	Nullable
sitter_conditions	_text	
view_count	int4	

Table applications

Name	Type	Constraints
id	uuid	Primary
request_id	uuid	
sitter_id	uuid	
message	text	Nullable
proposed_price	int4	Nullable
status	text	
created_at	timestampz	Nullable
updated_at	timestampz	Nullable

ERD-3

Table reservations

CHECK: (service_id IS NOT NULL OR request_id IS NOT NULL) AND (start_datetime < end_datetime)

Name	Type	Constraints
id	uuid	Primary
owner_id	uuid	
sitter_id	uuid	
service_id	uuid	Nullable
request_id	uuid	Nullable
start_datetime	timestampz	Nullable
end_datetime	timestampz	Nullable
total_price	int4	
status	text	
memo	text	Nullable
accepted_at	timestampz	Nullable
paid_at	timestampz	Nullable
started_at	timestampz	Nullable
completed_at	timestampz	Nullable
canceled_at	timestampz	Nullable
cancel_reason	text	Nullable
created_at	timestampz	Nullable
updated_at	timestampz	Nullable
application_id	uuid	Nullable

Table reservation_items

Name	Type	Constraints
id	uuid	Primary
reservation_id	uuid	
pet_id	uuid	

Table payments

CHECK: canceled_amount <= amount

Name	Type	Constraints
id	text	Primary
reservation_id	uuid	
owner_id	uuid	
sitter_id	uuid	
pay_method	text	
amount	int4	
canceled_amount	int4	
fee_rate	numeric	
platform_fee	int4	
settle_amount	int4	
status	text	
receipt_url	text	Nullable
auto_confirm_at	timestampz	Nullable
paid_at	timestampz	Nullable
canceled_at	timestampz	Nullable
settled_at	timestampz	Nullable
created_at	timestampz	Nullable
updated_at	timestampz	Nullable
payment_id	text	Unique

ERD-4

Table reviews

Name	Type	Constraints
id	uuid	Primary
reservation_id	uuid	Unique
owner_id	uuid	
sitter_id	uuid	
rating	int4	
content	text	
image_urls	_text	Nullable
created_at	timestampz	Nullable
updated_at	timestampz	Nullable
tags	_text	
detail_ratings	jsonb	

Table reports

Name	Type	Constraints
id	uuid	Primary
reporter_id	uuid	
target_type	text	
target_id	uuid	
reason	text	
content	text	Nullable
status	text	
handled_by	uuid	Nullable
handled_at	timestampz	Nullable
admin_memo	text	Nullable
created_at	timestampz	Nullable
updated_at	timestampz	Nullable
image_urls	_text	Nullable

Table notifications

Name	Type	Constraints
id	uuid	Primary
user_id	uuid	
type	text	
title	text	
content	text	
is_read	bool	
link_url	text	Nullable
created_at	timestampz	Nullable
updated_at	timestampz	Nullable

ERD-5

Table chat_rooms

Name	Type	Constraints
id	uuid	Primary
owner_id	uuid	
sitter_id	uuid	
room_type	text	
request_id	uuid	Nullable
application_id	uuid	Nullable
reservation_id	uuid	Nullable
created_at	timestampz	Nullable
updated_at	timestampz	Nullable
last_message	text	Nullable
last_message_at	timestampz	Nullable
owner_left	bool	
sitter_left	bool	

Table messages

Name	Type	Constraints
id	uuid	Primary
room_id	uuid	
sender_id	uuid	
content	text	
is_read	bool	
created_at	timestampz	Nullable
updated_at	timestampz	Nullable

Table extra_charges

시터의 추가금 요청 테이블

Name	Type	Constraints
id	uuid	Primary
reservation_id	uuid	
sitter_id	uuid	
owner_id	uuid	
amount	int4	
reason	text	
status	text	
payment_id	text	Nullable
requested_at	timestampz	Nullable
responded_at	timestampz	Nullable
created_at	timestampz	Nullable
updated_at	timestampz	Nullable

ERD-6

Table `care_records`

Name	Type	Constraints
<code>id</code>	<code>uuid</code>	Primary
<code>reservation_id</code>	<code>uuid</code>	
<code>sitter_id</code>	<code>uuid</code>	
<code>type</code>	<code>text</code>	
<code>service_type</code>	<code>text</code>	Nullable
<code>title</code>	<code>text</code>	
<code>status_text</code>	<code>text</code>	
<code>content</code>	<code>text</code>	
<code>fields</code>	<code>jsonb</code>	
<code>image_urls</code>	<code>_text</code>	
<code>created_at</code>	<code>timestampz</code>	

Table `bank_accounts`

Name	Type	Constraints
<code>id</code>	<code>uuid</code>	Primary
<code>user_id</code>	<code>uuid</code>	Unique
<code>bank_name</code>	<code>text</code>	
<code>account_number</code>	<code>text</code>	
<code>account_holder</code>	<code>text</code>	
<code>created_at</code>	<code>timestampz</code>	
<code>updated_at</code>	<code>timestampz</code>	

02

주요 기능 시연

보호자가 펫시터를 찾아 예약·결제하고, 서비스가 완료되기까지

01

온보딩 — 회원가입·본인인증

가입 후 본인인증을 거쳐 사용자를 인증합니다.

02

게시판 — 글쓰기 → 상세 → 지원하기

보호자가 돌봄을 구인하고, 펫시터가 지원합니다

03

펫시터 찾기 — 검색 → 상세 → 예약하기

보호자의 위치를 기준으로 지도에서 펫시터를 찾아 예약을 신청합니다

04

채팅 — 예약 확정 → 결제 → 서비스 시작 → 완료

매칭부터 서비스 종료, 후기 작성 페이지 이동까지 채팅 한 화면에서 처리됩니다

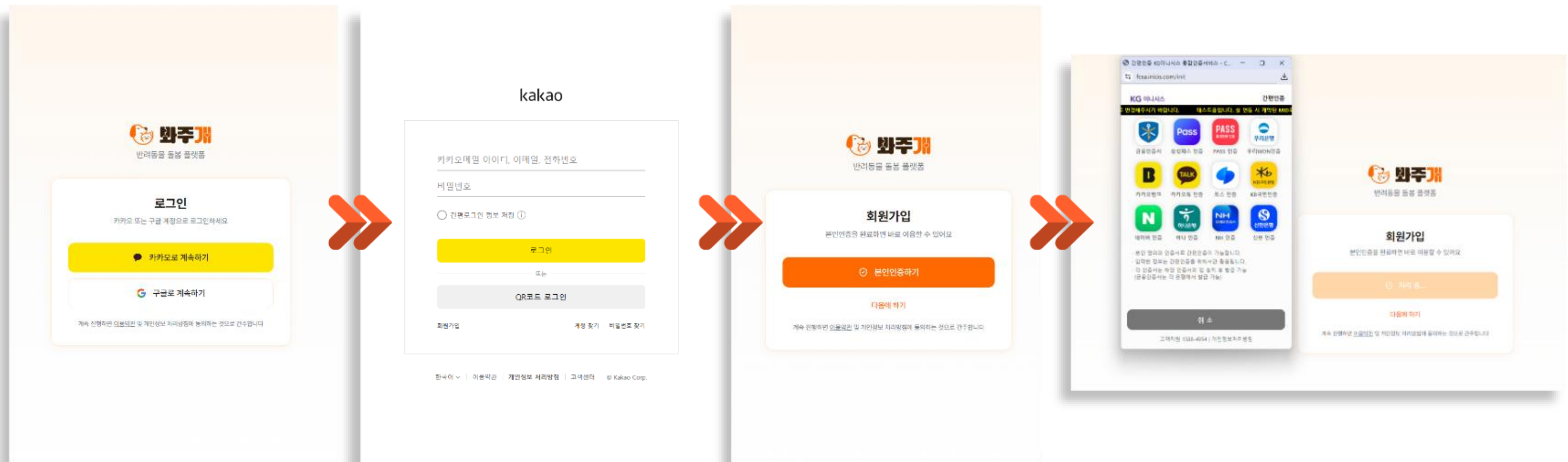
05

프로필 — 반려동물 등록·예약 내역·게시글 관리·후기관리·수익 확인

보호자·펫시터 각 역할의 누적 데이터를 확인합니다

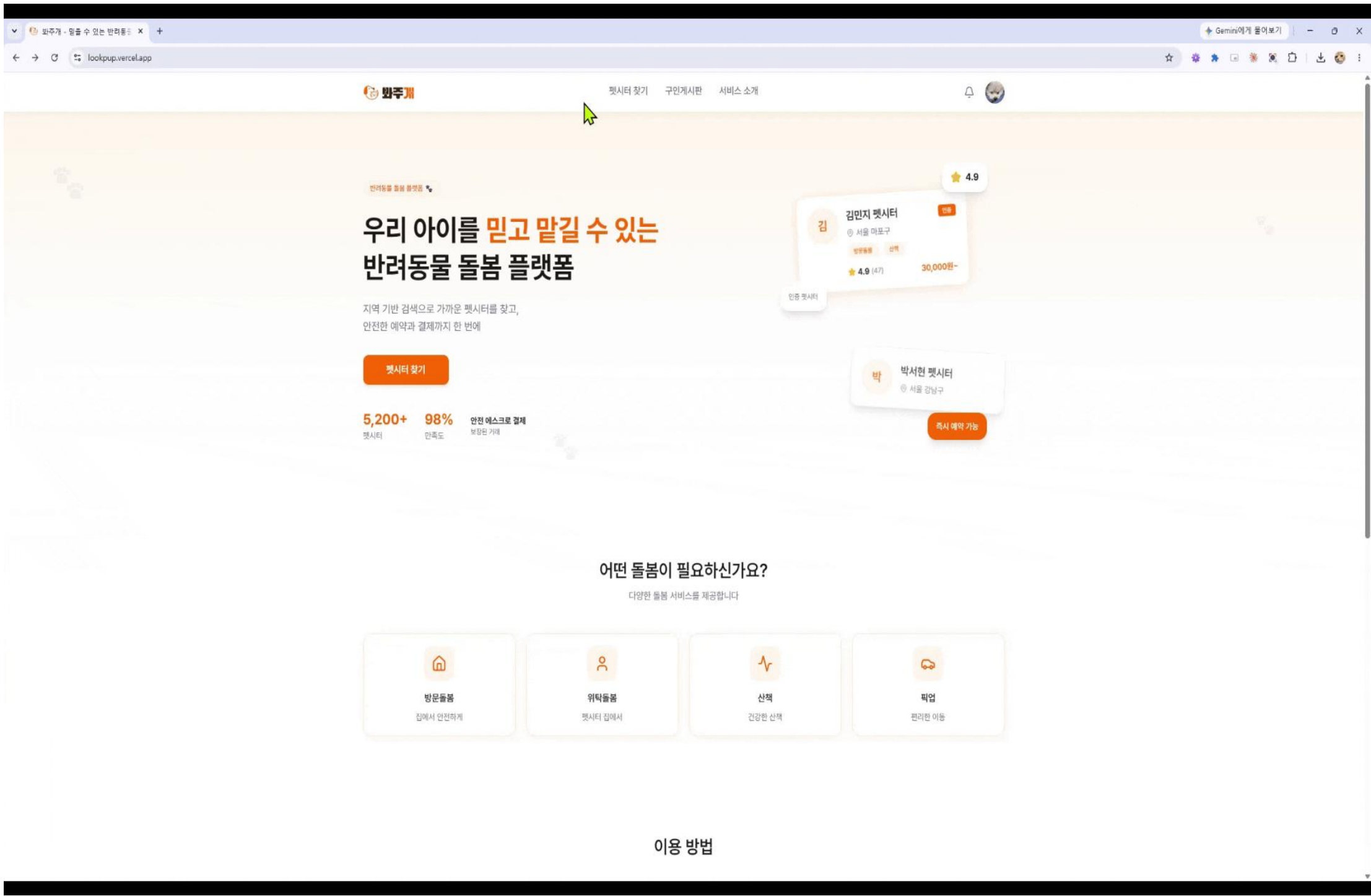
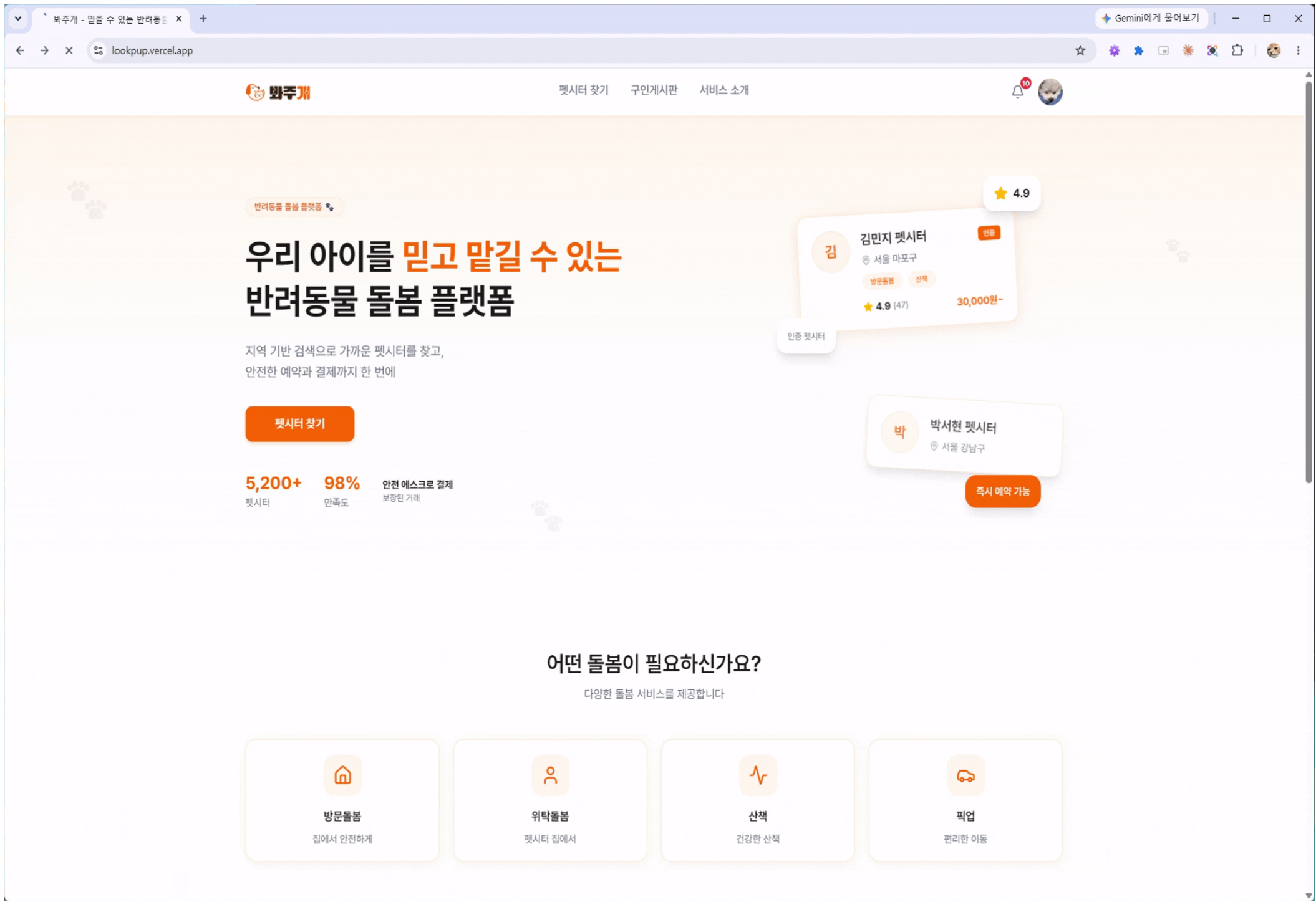
소셜 로그인부터 본인인증까지

카카오·구글 소셜 로그인으로 가입한 뒤, 펫시터 등록 등 신뢰가 필요한 활동을 시작하려면 PortOne 본인인증을 거칩니다.



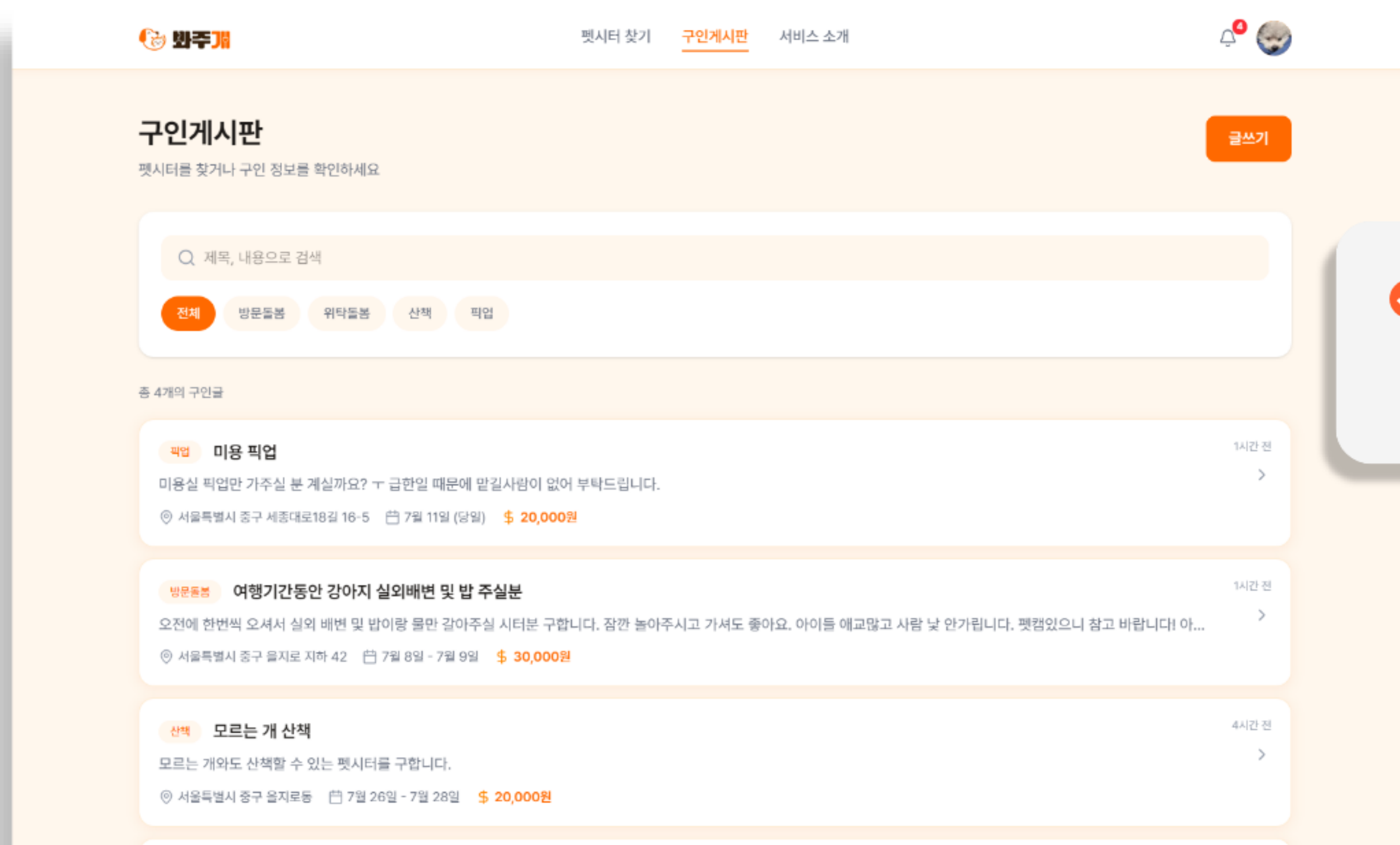
봐주개

home화면에서 서비스를 간편하게 이용할 수 있습니다.



구인 글을 필터·검색으로 탐색

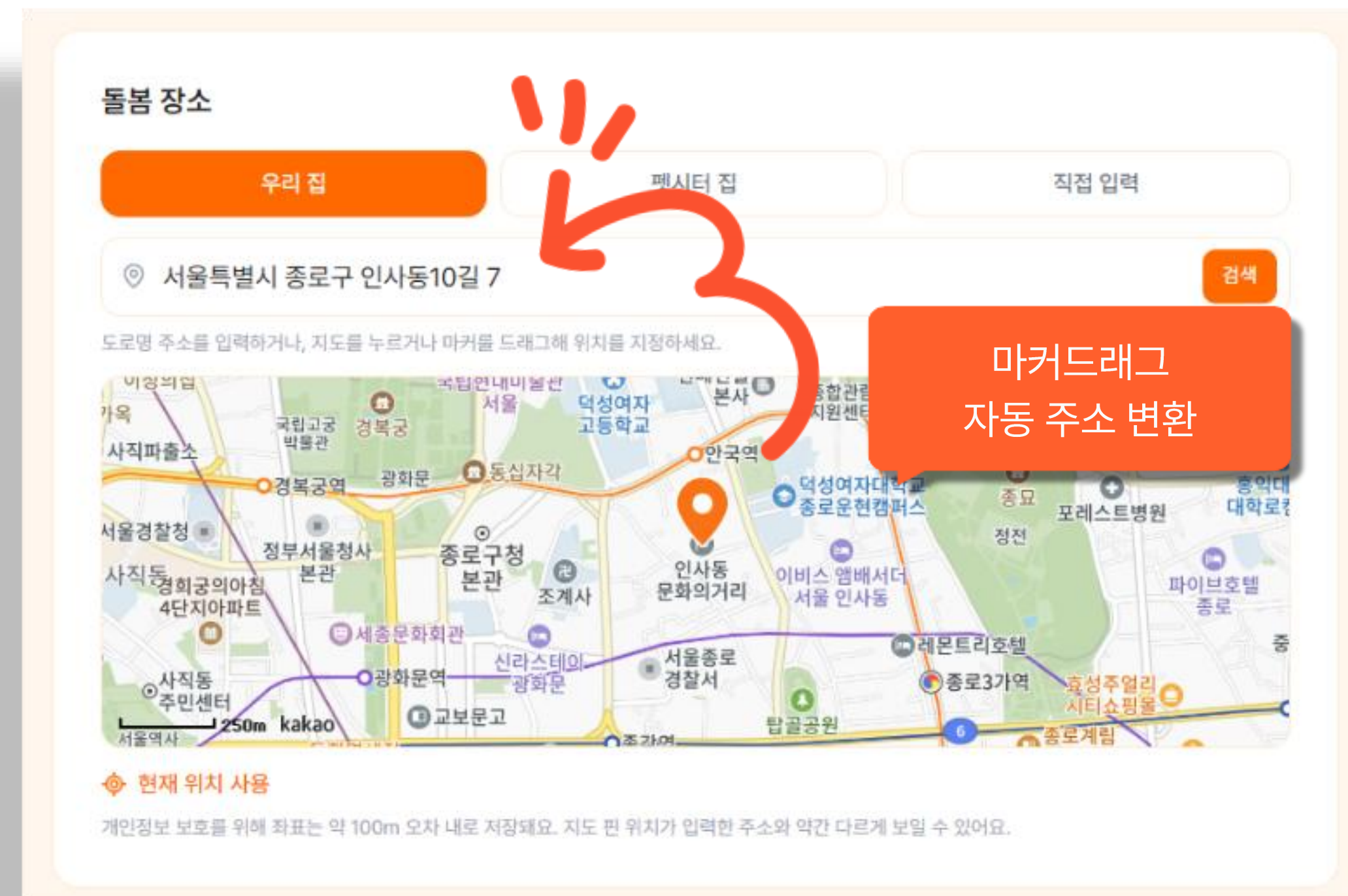
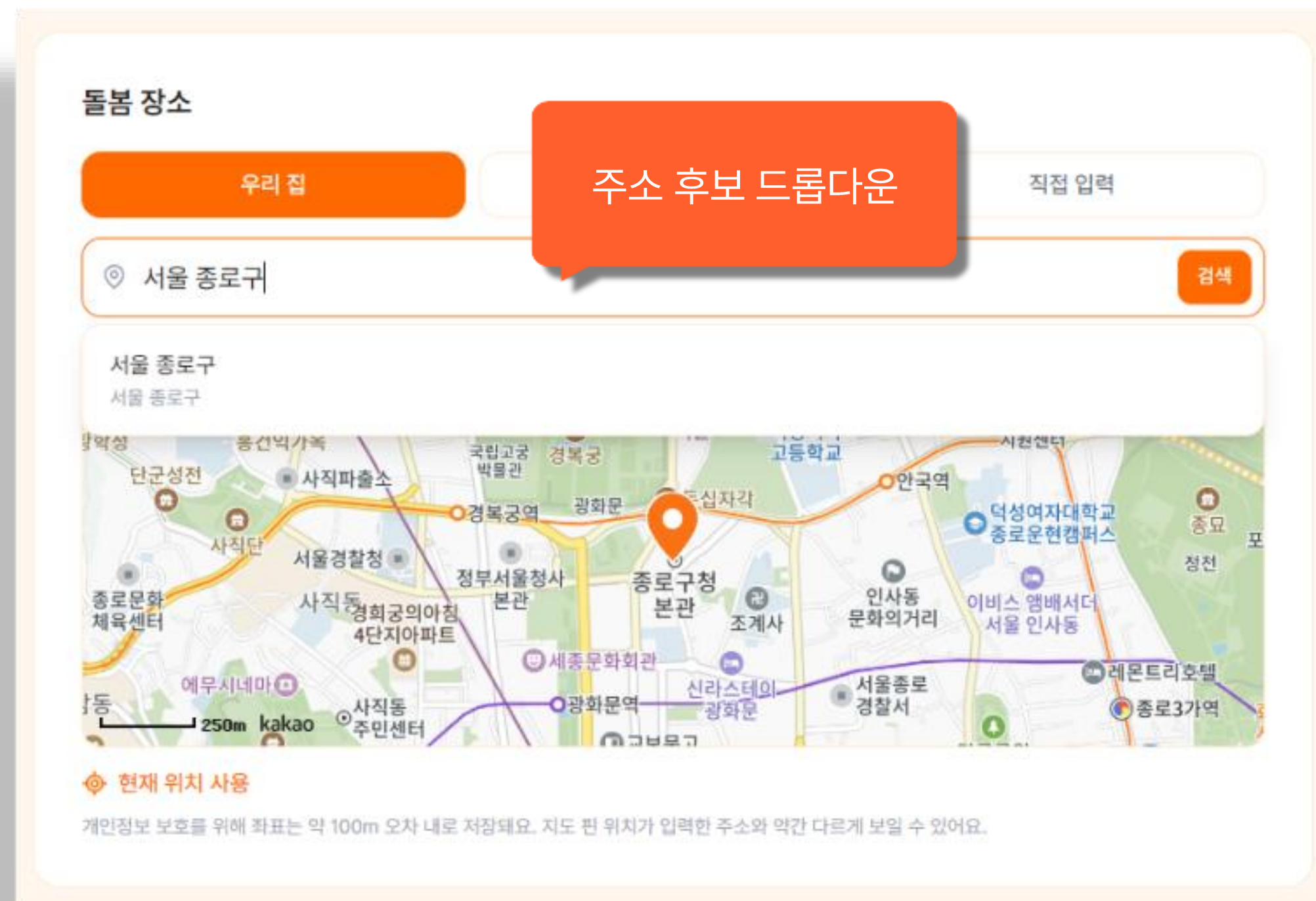
텍스트 검색 — 제목·내용 기준 | 페이지네이션 — 5개씩 표시



- ✓ 카테고리 필터 — 방문돌봄 / 위탁돌봄 / 산책 / 픽업
- 텍스트 검색 — 제목·내용 기준
- 페이지네이션 — 5개씩 표시

카카오맵 주소 자동완성 · 역지오코딩

입력값이 바뀔 때마다 300ms 디바운스 후 카카오 Geocoder API를 호출해 주소 후보를 드롭다운으로 보여줍니다.
지도를 클릭하거나 마커를 드래그하면 좌표 → 주소로 자동 변환됩니다.



게시글 작성 → 구인글 지원하기

The screenshot shows a web browser window with the URL `http://localhost:3000/board/write`. The page title is "보호자 - 게시글 작성" (Guardian - Posting). The main heading is "돌봄 요청 게시글 작성" (Posting a Request for Help). There are two buttons at the top right: "임시저장" (Save Draft) and "게시하기" (Post).

The form asks "어떤 돌봄이 필요하신가요?" (What kind of help do you need?). It has four options: "방문 돌봄" (Home Visit), "위탁 돌봄" (Foster Care), "산책" (Walking), and "픽업" (Pickup).

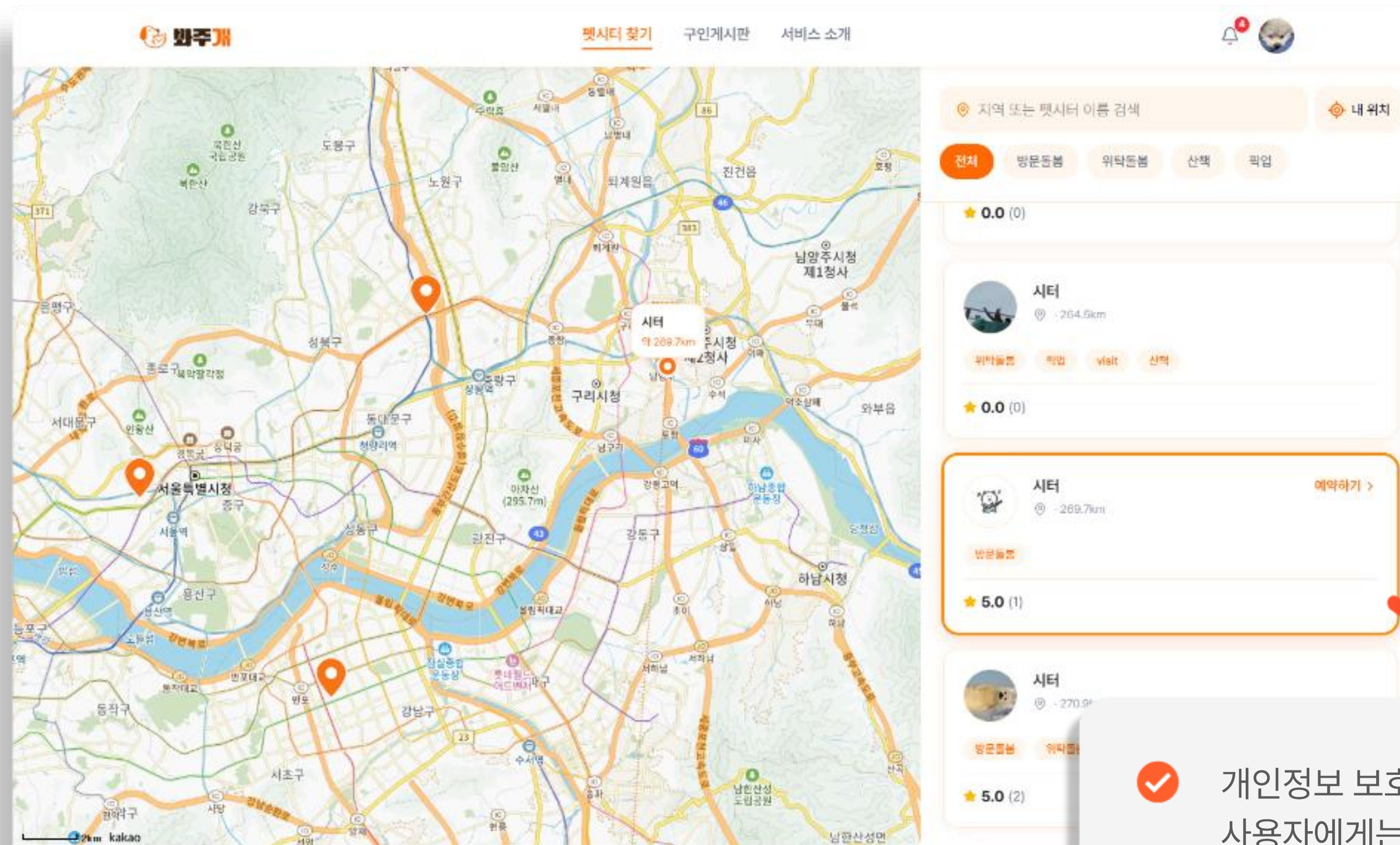
Below the options, it asks "예산을 입력해주세요" (Please enter your budget). There is a text input field with "0" and a unit "원" (won). Below the input field are five buttons: "10,000원", "20,000원", "30,000원", "50,000원", and "협의 가능" (Negotiable).

At the bottom, there is a section for "날짜 · 시간" (Date · Time). It shows a date picker for "2026년 7월" (July 2026).

At the bottom right, there is a "게시하기" (Post) button.

어떤 돌봄이 필요하신가요?

카카오맵 위에 펫시터 마커를 표시하고, 리스트 카드를 클릭하면 지도 핀이 강조되며 "예약하기" CTA가 노출됩니다. 현재 위치 기준 실거리는 Haversine 공식으로 계산합니다.



✓ 확인 정보

프로필, 제공 서비스 목록, 평점, 리뷰

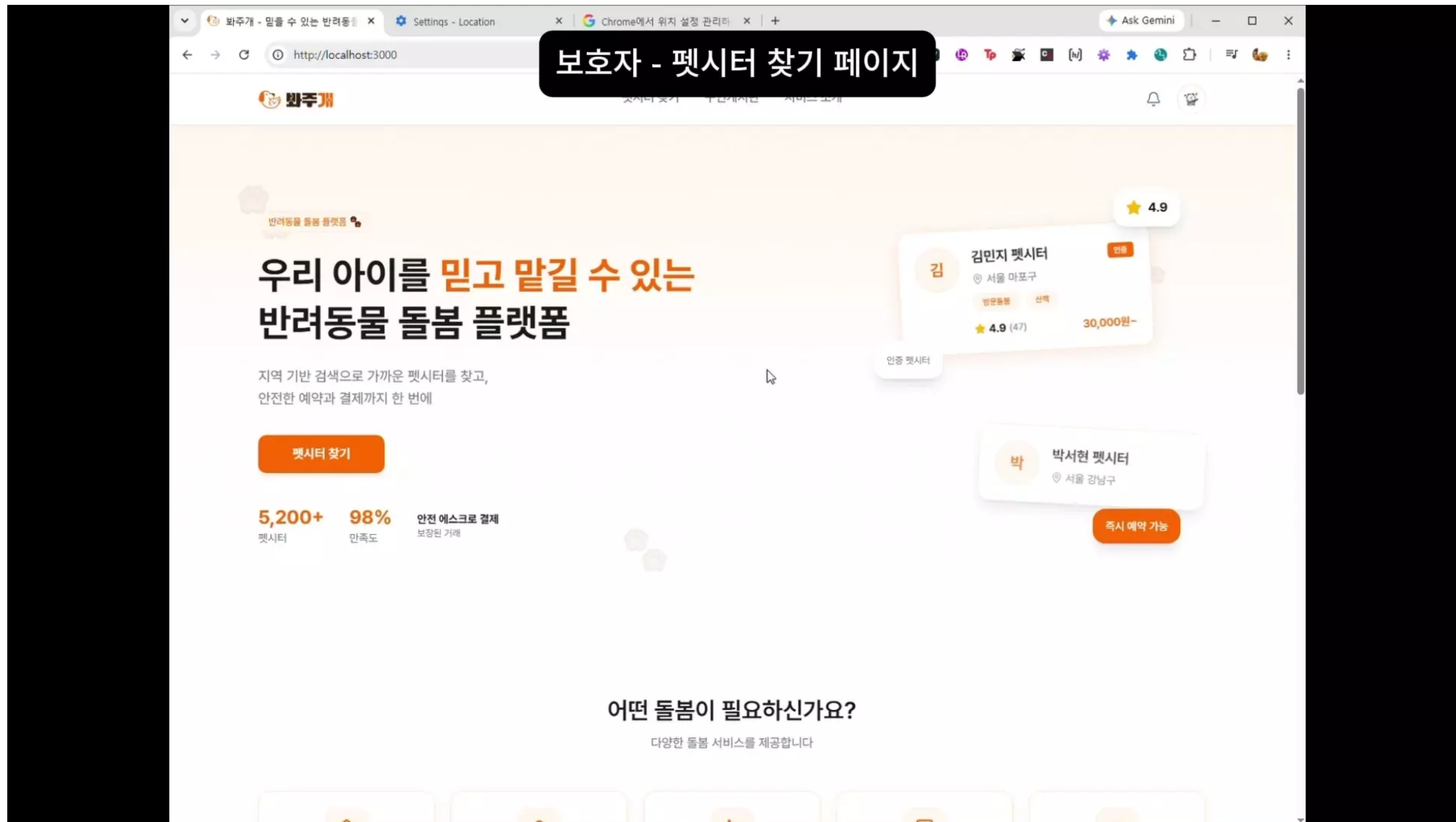
✓ 예약 신청

"예약 신청" 버튼 클릭 → 날짜·시간·반려동물 선택 단계로 진행

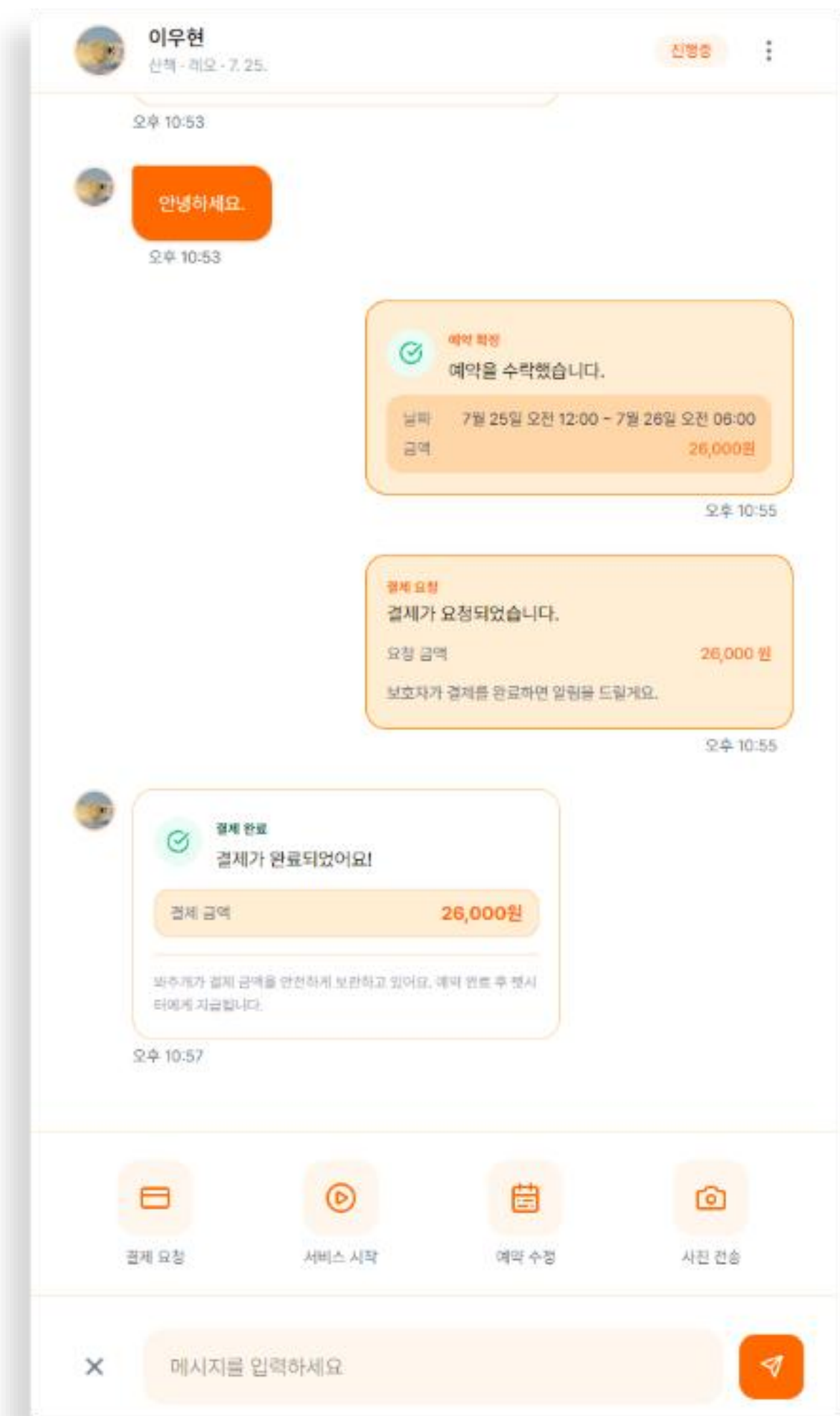


개인정보 보호를 위해 좌표값은 100m오차 내로 저장되어 다른 사용자에게는 튜닝된 정보값이 나타납니다.

검색 → 상세 프로필 → 예약하기



3개 탭 안에서 끝나는 예약의 전 과정



- 01 펫시터가 결제 요청 메시지 발송
- 02 보호자가 채팅 내에서 바로 카드 결제 (tosspayments)
- 03 펫시터 → 예약 날짜에 맞춰 "서비스 시작"
- 04 펫시터 → 돌봄 기록 전송 (사진 포함)
- 05 펫시터 → "서비스 완료"
- 06 보호자 → 완료 확인 → 리뷰 작성 이동

탭	역할
1:1 채팅	서비스 진행 채팅 (결제 · 돌봄 기록 · 사진 전송)
예약 목록	펫시터가 보호자의 예약 수락 / 거절
지원 목록	보호자가 펫시터의 지원 선택



지원자 확정 → 결제 → 서비스 진행 → 완료

The screenshot shows the homepage of the '반려동물 돌봄 플랫폼' (Pet Care Platform). At the top, there's a navigation bar with '반주개' logo, '펫시터 찾기' (Find Pet Sitter), '구인게시판' (Job Posting Board), and '서비스 소개' (Service Introduction). A green cursor points to the '서비스 소개' link. A black callout box with white text says '▲ 펫시터 - 서비스 예약 요청 알림' (Pet Sitter - Service Reservation Request Notification). The main content area features the headline '우리 아이를 믿고 맡길 수 있는 반려동물 돌봄 플랫폼' (Pet Care Platform you can trust with your child) and a sub-headline '지역 기반 검색으로 가까운 펫시터를 찾고, 안전한 예약과 결제까지 한 번에' (Find nearby pet sitters based on location, and complete safe reservations and payments all at once). Below this is an orange button labeled '펫시터 찾기' (Find Pet Sitter). To the right, there are two pet sitter profiles: '김민지 펫시터' (Kim Minji Pet Sitter) with a 4.9 rating and '30,000원-' fee, and '박서현 펫시터' (Park Sehyun Pet Sitter) with a '즉시 예약 가능' (Available for immediate booking) button. At the bottom, it asks '어떤 돌봄이 필요하신가요?' (What kind of care do you need?) and mentions '다양한 돌봄 서비스를 제공합니다' (We provide various care services).

반주개

펫시터 찾기 구인게시판 서비스 소개

반려동물 돌봄 플랫폼

▲ 펫시터 - 서비스 예약 요청 알림

우리 아이를 믿고 맡길 수 있는
반려동물 돌봄 플랫폼

지역 기반 검색으로 가까운 펫시터를 찾고,
안전한 예약과 결제까지 한 번에

펫시터 찾기

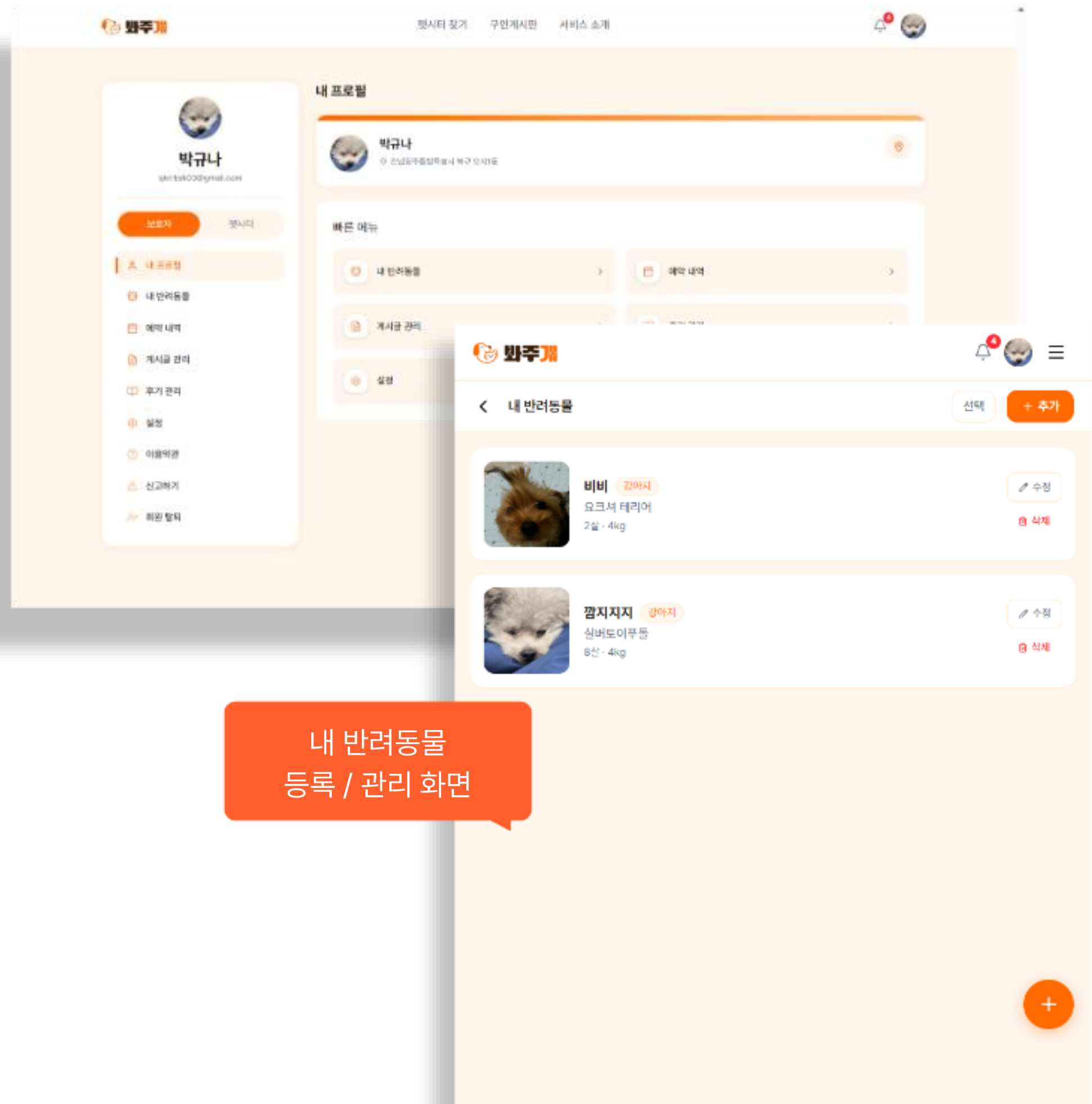
5,200+ 펫시터 98% 만족도 안전 에스스로 결제
보장된 거래

김민지 펫시터
서울 마포구
방문돌봄 산책
★ 4.9 (47) 30,000원-
인증 펫시터

박서현 펫시터
서울 강남구
즉시 예약 가능

어떤 돌봄이 필요하신가요?
다양한 돌봄 서비스를 제공합니다

하나의 계정, 두 개의 역할



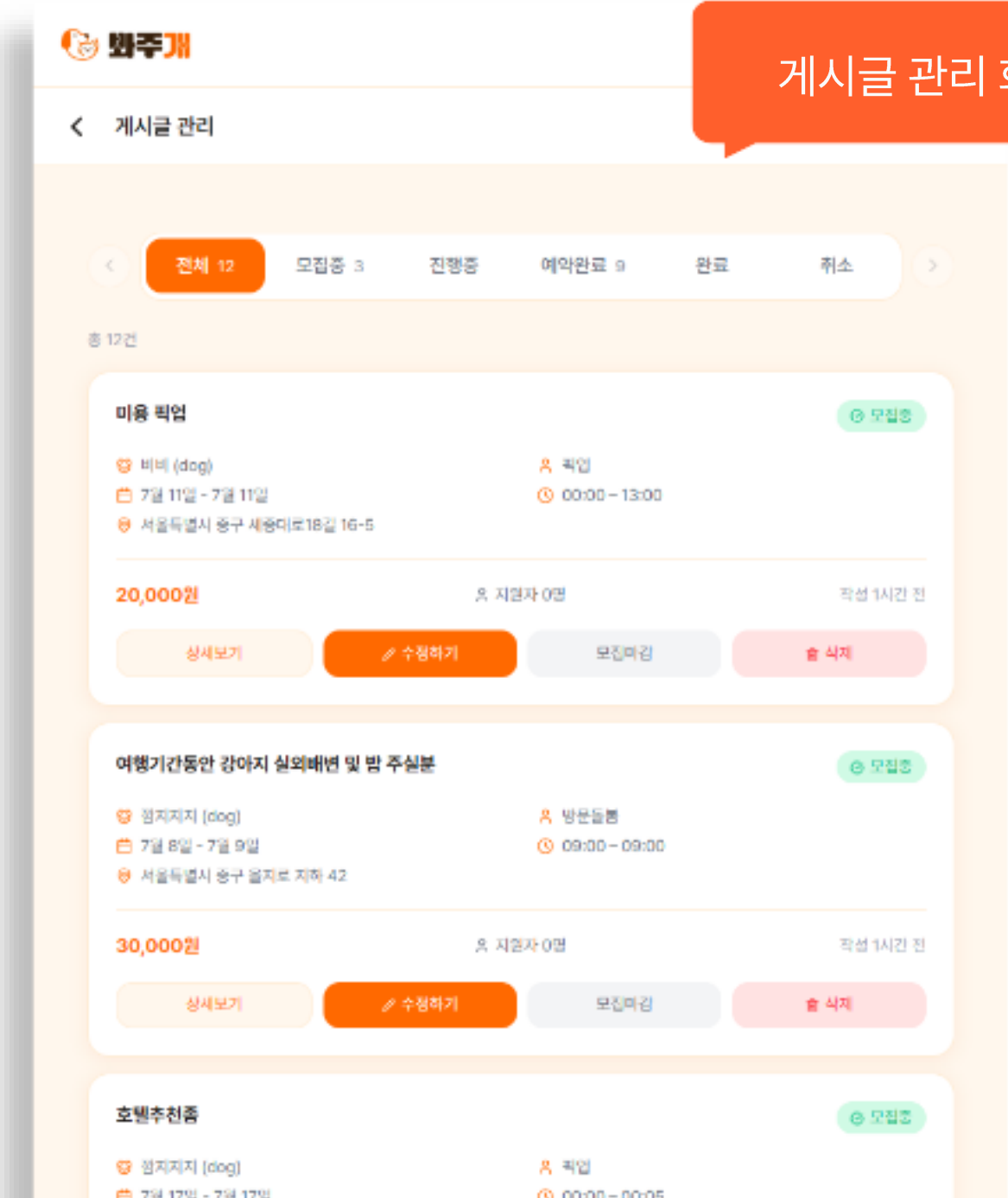
내 반려동물
등록 / 관리 화면

✓ **보호자 화면**
예약 내역 확인, 내 반려동물 등록·관리, 게시글 관리, 후기 관리 등등

예약 내역 화면

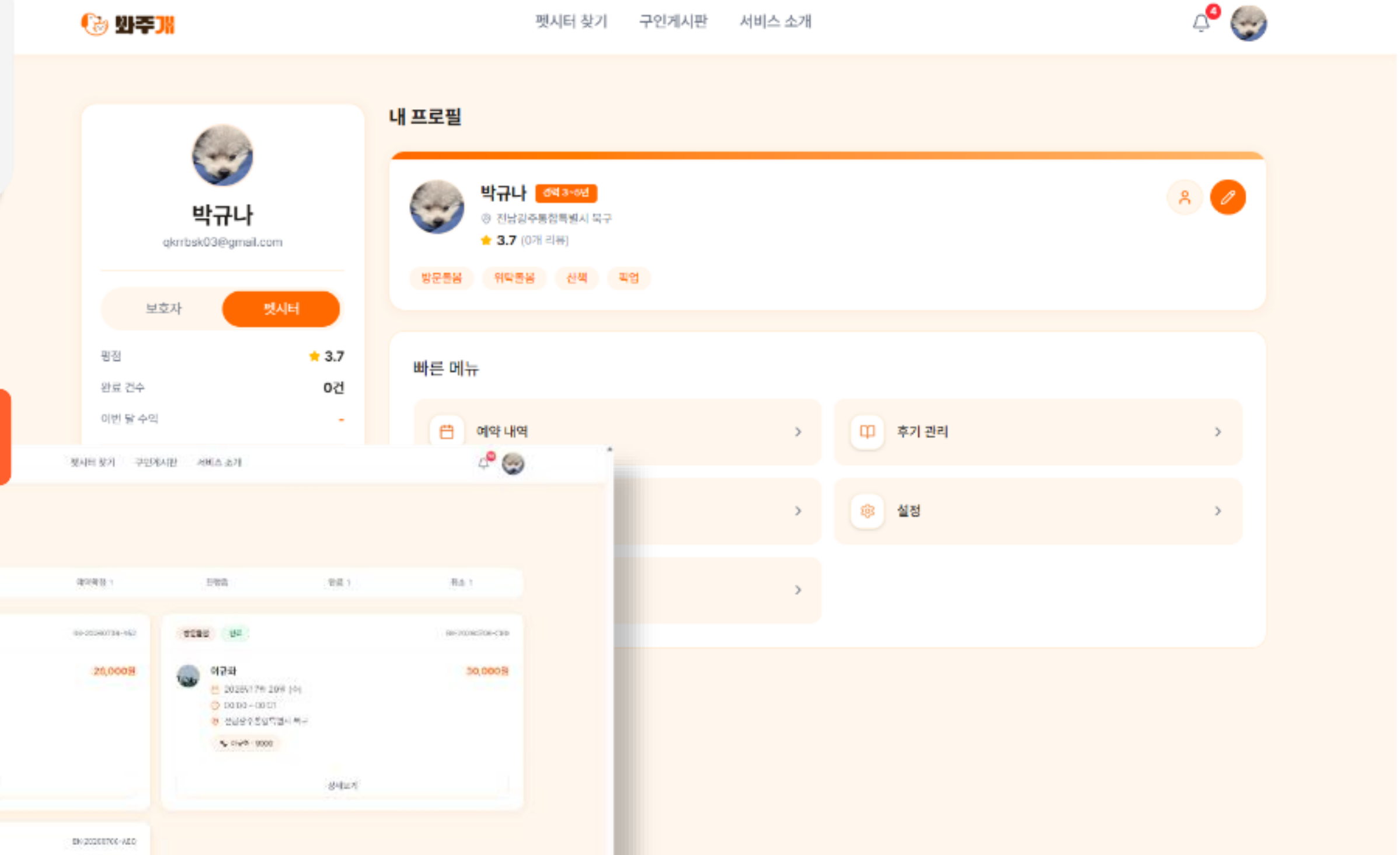


게시글 관리 화면

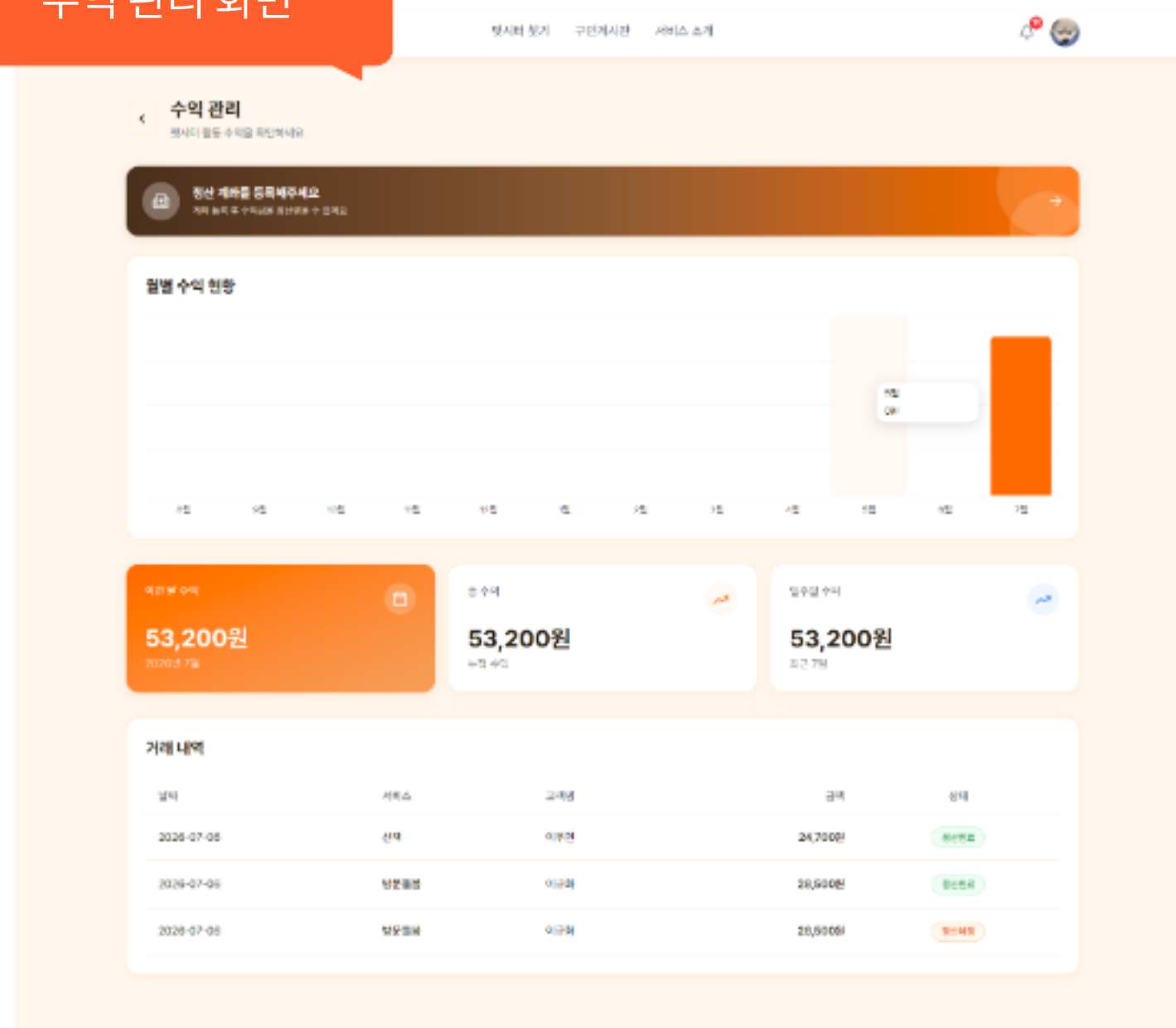


하나의 계정, 두 개의 역할

- ✓ **펫시터 화면**
시터 프로필 등록 후 기간별 수익 차트와 거래 내역 확인



수익 관리 화면

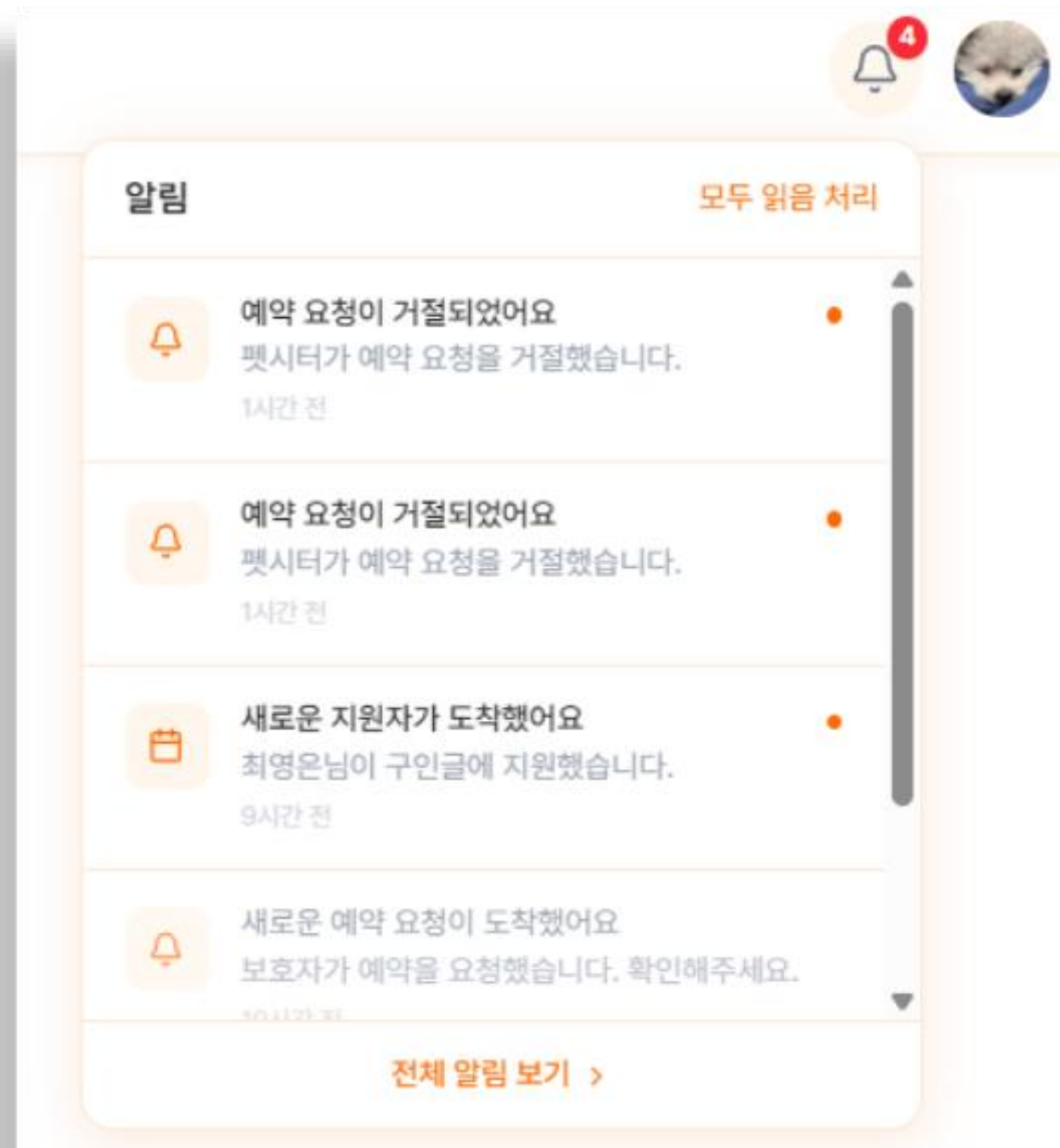


예약 내역 화면

예약 상태	예약 일자	예약 시간	예약 금액
신청	2025년 7월 25일 (토)	02:00 ~ 05:00	24,700원
방문통보	2025년 7월 25일 (토)	02:00 ~ 05:00	28,500원
위탁통보	2025년 7월 25일 (토)	02:00 ~ 05:00	20,000원

DB 변경을 직접 구독하는 실시간 알림

채팅의 Broadcast와 달리, 알림은 notifications 테이블에 row가 INSERT되는 순간을 postgres_changes로 구독해 새로고침 없이 전달합니다



신고 처리와 예약 현황 모니터링



신고 관리

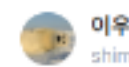
접수 → 확인 → 유저 정지 처리까지 대시보드에서 일괄 처리

신고 관리 예약 상태 관리 댓글 승인

신고 관리

총 2건의 신고가 접수되었습니다.

전체 2 대기 2 처리중 0 완료 0 반려 0



이우현
shimshinhan@kakao.com

댓글 신고 기타

07.08 17:37 대기



박규나
qkrbsk03@gmail.com

댓글 신고 부적절한 언행

07.01 13:53 대기

신고 사유 부적절한 언행
대상 유형 댓글 신고
대상 ID 74c f5cab - 7ddb - 45e7 - a7be - a7ec40e5c68c
상세 내용

말이 너무 너무 너무너무 안이빠요

증거 사진



현재 상태 대기

관리 액션

임시 정지

7 일

정지

정지 해제

정지 해제

댓글 신고 권한 강등

댓글 신고 권한 강등

처리하기

신고 관리 예약 상태 관리 댓글 승인

신고 관리

총 2건의 신고가 접수되었습니다.

전체 2 대기 2 처리중 0 완료 0 반려 0



이우현
shimshinhan@kakao.com

댓글 신고 기타

07.08 17:37 대기



박규나
qkrbsk03@gmail.com

댓글 신고 부적절한 언행

07.01 13:53 대기

신고 사유 부적절한 언행
대상 유형 댓글 신고
대상 ID 74c f5cab - 7ddb - 45e7 - a7be - a7ec40e5c68c
상세 내용

말이 너무 너무 너무너무 안이빠요

증거 사진



현재 상태 대기

관리 액션

임시 정지

7 일

정지

정지 해제

정지 해제

댓글 신고 권한 강등

댓글 신고 권한 강등

처리하기



예약 상태 모니터링

전체 예약 현황을 실시간으로 확인

승인된 펫시터

신고 관리

예약 상태 관리

펫시터 승인

펫시터 승인 관리

총 6건의 펫시터 신청이 있습니다.

전체 6

대기 1

승인 5

반려 0

펫시터 ID

신청자

연락처

활동 지역

경력

기본 가격

서비스

가능 동물

소개

514c39ea-4d20-4309-af8b-6f1a5cac952e

최영은 (0sliverchoi321z@gmail.com)

01062472653

경기 구리시

없음

0원

방문돌봄

고양이

초보 펫시터입니다
성실하고 꼼꼼하게 임하겠습니다

현재 상태

대기

상태 변경

대기 (현재)

승인

반려

07.07 10:25

대기

03

Before / After — 트러블슈팅

예약 금액이 이용 일수를 반영하지 못한 문제

BEFORE

며칠을 예약하든 `total_price = service.price`, 즉 하루 단가만 청구되어 장기 예약일수록 시터가 손해를 봄

AFTER

KST 기준 달력일을 계산해 단가 × 이용 일수로 산정. UI의 "박(nights)" 표기도 "일(days)" 기준으로 통일

```
//src/app/actions/reservations.ts
```

```
const DAY_MS = 24 * 60 * 60 * 1000;
+ const KST_OFFSET_MS = 9 * 60 * 60 * 1000; // UTC+9
+ const toKstDayIndex = (iso) =>
+   Math.floor((new Date(iso).getTime() + KST_OFFSET_MS) / DAY_MS);
+ const days = Math.max(1, toKstDayIndex(end) - toKstDayIndex(start) + 1);
+ const totalPrice = service.price * days;
- total_price: service.price,
+ total_price: totalPrice,
```



모든 신규 예약의 결제 금액이 실제 이용 기간에 맞게 산정

RLS 정책 오류로 상대 메시지가 실시간으로 보이지 않던 문제

BEFORE

messages 테이블 RLS가 chat_rooms.sitter_id = auth.uid()로 비교됐지만, sitter_id는 sitters 테이블 row ID이고 auth.uid()는 인증 UUID라 값이 불일치. 펫시터는 SELECT 권한을 얻지 못해 postgres_changes 이벤트를 수신하지 못함

AFTER

postgres_changes를 broadcast 방식으로 전환. 발신자는 전송 성공 시 addMessage로 본인 화면에 즉시 반영되고, broadcastMessage 함수로 채널에 보내줌, 수신자는 broadcast 이벤트 구독으로 즉시 수신할 수 있게 됨. 이는 권한 불일치 문제와 무관하게 양방향 실시간 수신이 보장됨.

```
// ChatClient.tsx
const deliverMessage = useCallback(
  (roomId: string, message) => {
    if (activeRoomIdRef.current === roomId) addMessage(message);
    broadcastMessage(roomId, message);
  },
  [addMessage, broadcastMessage],
);
```

발신자: 전송 성공 시
즉시 반영 + 채널 broadcast

```
const broadcastMessage = useCallback((roomId, m) => {
  if (roomId === activeRoomId) {
    channelRef.current?.send({ type: "broadcast", event: "new_message", payload: m });
    return;
  }
  // 비활성 방인 경우 임시 채널을 열어 전송 후 정리
  const channel = supabase.channel(`room-${roomId}`);
  channel.subscribe((status) => {
    if (status !== "SUBSCRIBED") return;
    channel.send({ type: "broadcast", event: "new_message", payload: m })
      .finally(() => supabase.removeChannel(channel));
  });
}, [activeRoomId]);
```

broadcastMessage
DB 저장 결과를 채널로 전송

```
const channel = supabase
  .channel(`room-${activeRoomId}`)
  .on("broadcast", { event: "new_message" }, ({ payload }) => {
    const m = payload as MessageApiItem;
    if (m.sender_id === userId) return;
    setMessages((prev) =>
      prev.some((msg) => msg.id === m.id) ? prev : [...prev, toMessage(m, userId)],
    );
  })
  .subscribe();
```

수신자: broadcast 이벤트 구독

내가 보낸 메시지로 내 화면의 목록 미리보기가 안 바뀌던 문제

BEFORE

채팅 목록의 "마지막 메시지 미리보기"가 수신자 화면에서는 정상 갱신되지만, 발신자 화면에서는 갱신되지 않음. Realtime broadcast 채널은 발신자 자신에게는 자신이 보낸 이벤트를 되돌려주지 않기 때문

AFTER

메시지 전송 성공 응답을 받은 시점에 broadcast 수신을 기다리지 않고, 클라이언트에서 곧바로 `updatePreview()`를 호출해 목록 상태를 갱신

전송 성공 직후
`updatePreview()` 직접 호출

```
// src/app/chat/page.tsx
if (result.data) {
  addMessage(result.data);
  broadcastMessage(result.data);
  // broadcast는 self=false라 내 구독에 안 옴 → 직접 미리보기 갱신
  updatePreview(activeRoomId, result.data.content, result.data.created_at);
}
```

```
// src/hooks/chat/useChatRooms.ts
function updateRoomPreview(roomId: string, content: string,
  createdAt: string) {
  const time = formatTime(createdAt);
  setRooms((prev) =>
    prev.map((r) => (r.id === roomId ? { ...r, lastMessage: content,
    time } : r)),
  );
  setApplicants((prev) =>
    prev.map((a) => (a.id === roomId ? { ...a, preview: content } :
    a)),
  );
}
```

공개 검색 RPC의 개인정보 노출

BEFORE

펫시터 목록·지도 검색에 쓰이는 공개 RPC `get\_petsitters\_filtered` (Supabase DB 함수, 로그인 없이 anon 키로 직접 호출 가능)를 비로그인 사용자가 RPC를 직접 호출하면(devtools/curl 등) 지도 UI를 거치지 않고 반환 가능 → 집 위치 특정 가능

AFTER

full_name → display_name으로 대체 (실명 대신 표시용 이름)
user_id(인증 UUID) 컬럼 자체를 응답에서 제거
주소를 시/구/동 단위로 축약한 display_area만 반환 (도로명·상세주소 제거)
좌표를 소수점 3자리로 반올림해 반환 (오차 ≈ 100m)



저장 시점에도 좌표를 뭉개어 원본 정밀도 자체를 남기지 않음

```
//src/utils/geoPrivacy.ts
const PRECISION = 3;

export function fuzzCoordinate(value: number): number {
  return Math.round(value * 10 ** PRECISION) / 10 ** PRECISION;
}
```

```
// src/app/actions/sitters.ts (createSitter)
.insert({
  ...sitterFields,
  latitude: fuzzCoordinate(sitterFields.latitude),
  longitude: fuzzCoordinate(sitterFields.longitude),
  user_id: user.id,
})
```

동시에 채팅을 시작하면 1:1 채팅방이 2개 생성되던 문제

BEFORE

기존에는 '1:1 채팅방의 생성 시 owner_id=나, sitter_id=의 상대 단방향으로만 기존 방을 SELECT로 확인
→ 없으면 INSERT(check-then-act)의 방식으로 구현

각자 펫시터 프로필이 있는 두 사용자가 거의 동시에 채팅방 생성을 요청하면 서로의 INSERT가 반영되기 전에 양쪽 다 "기존 방 없음"으로 판단하여 동일한 두 사용자 사이에 방이 2개 생성.

AFTER

상대방의 실제 user_id와 내 sitters.id를 먼저 병렬 조회(상대가 시터인 경우, 펫시터인 경우 전부 확인)한 뒤, 양방향으로 기존 방을 SELECT로 확인하도록 개선함. 두 조합을 or로 한번에 조회하고, 누가 먼저 채팅방 생성을 요청하였더라도 기존 방을 찾아 재사용

✓ 조회 조건을 양방향으로

```
const [{ data: otherSitter }, { data: mySitter }] = await Promise.all([
  db.from("sitters").select("user_id").eq("id", input.sitter_id).maybeSingle(),
  db.from("sitters").select("id").eq("user_id", user.id).maybeSingle(),
]);

const otherUserId = otherSitter?.user_id;
const mySitterId = mySitter?.id;

if (otherUserId && mySitterId) {
  const { data } = await db
    .from("chat_rooms")
    .select("id")
    .eq("room_type", "direct")
    .or(
      `and(owner_id.eq.${user.id},sitter_id.eq.${input.sitter_id}),` +
      `and(owner_id.eq.${otherUserId},sitter_id.eq.${mySitterId})`,
    )
    .maybeSingle();
  existingRoom = data;
} else {
  // 상대가 펫시터가 아닌 경우 기존 단방향 조회로 풀백
  ...
}
```

실시간 채팅에서 마주친 자잘한 정합성 문제 3가지

스크롤 튜

이전 메시지를 불러오면 스크롤이 최상단으로 튜는 문제. 로드 전 scrollHeight를 앵커로 저장해두고, useLayoutEffect에서 늘어난 높이만큼 scrollTop을 보정

메시지 중복 표시

빠른 수신을 위해 broadcast와, 놓치지 않기 위해 postgres_changes를 동시에 구독하다 보니 같은 메시지가 두 번 렌더링됨. 메시지 ID로 중복 체크 후 무시하도록 수정

방 전환 Race Condition

채팅방을 빠르게 전환하면 이전 방의 fetch 응답이 늦게 도착해 현재 방 상태를 덮어씀. AbortController로 방 전환 시 이전 요청을 즉시 취소



실시간 채널을 이중으로 쓸 땐 항상 "중복 수신"과 "취소 가능성"을 함께 설계해야 한다.

모바일에서 **tosspayments** 결제창 잘림 문제

BEFORE

PortOne SDK는 데스크톱에서 팝업으로 결제창을 띄우지만, 모바일에서는 팝업 차단 정책으로 PG사 페이지로 리다이렉트. 복귀 후 결제 완료를 처리해줄 페이지가 없었음

AFTER

/payment/complete를 returnUrl로 지정. 쿼리 파라미터로 전달되는 paymentId(성공) 또는 code(실패)를 읽어 결과를 표시



모바일 결제 연동 시 팝업 방식만 가정하지 말고 리다이렉트 착지 페이지를 항상 함께 준비해야 한다.

클라이언트가 결제 금액을 임의로 지정할 수 있던 문제

BEFORE

createPayment가 클라이언트로부터 requestedAmount를 그대로 받아 결제 금액으로 사용
— 요청을 조작하면 1원 결제로 예약을 확정할 수 있는 취약점

AFTER

결제 금액 파라미터 자체를 제거하고, 서버가 reservation.total_price만을 신뢰 가능한 금액으로 사용하도록 변경

```
export async function createPayment(
  reservationId: string,
  payMethod: "CARD" | "VIRTUAL_ACCOUNT" | "TRANSFER",
  - requestedAmount?: number,
) {
  ...
  - const amount = requestedAmount ?? reservation.total_price;
  + const amount = reservation.total_price;
  - if (requestedAmount && requestedAmount !== reservation.total_price) {
  -   await db.from("reservations").update({ total_price: requestedAmount }).eq("id", reservationId);
  - }
```



결제 금액처럼 신뢰가 중요한 값은 클라이언트 입력을 절대 그대로 신뢰하지 않고, 서버가 보유한 값만 사용해야 한다.

예약 시 타인의 반려동물 ID를 끼워 넣을 수 있던 문제

BEFORE

예약 요청의 pet_ids가 실제로 요청자 소유인지 검증하지 않아,
다른 유저의 반려동물 ID를 임의로 넣어 예약할 수 있었음

AFTER

verifyPetOwnership 헬퍼로 pet_ids 전부가 owner_id = 요청자인지
개수 비교로 확인 후 통과한 경우에만 예약 생성

```
+ async function verifyPetOwnership(db, petIds: string[], userId: string) {  
+   const { count } = await db  
+     .from("pets")  
+     .select("id", { count: "exact", head: true })  
+     .in("id", petIds)  
+     .eq("owner_id", userId)  
+     .is("deleted_at", null);  
+   return (count ?? 0) === new Set(petIds).size;  
+ }  
  
+ if (!(await verifyPetOwnership(db, input.pet_ids, user.id))) {  
+   return { error: { code: "FORBIDDEN", message: "본인의 반려동물만 예약에 추가할 수 있습니다." } };  
+ }
```



배열 형태 입력값(pet_ids)은 개수 일치만으로도
"요청한 ID 전부가 소유물인지"를 저비용으로 검증할 수 있다.

RSC / RCC 미분리 문제

AFTER

RSC와 RCC분리를 통한 SEO 성능 개선

page.tsx

```
//src/app/myprofile/page.tsx

import { getServerUser } from "@utils/supabase/serverUser";
import { createServiceClient } from "@utils/supabase/service";
import { getMySitterProfile } from "@app/actions/sitters";
import MyProfileClient from "@components/myprofile/MyProfileClient";
import type { UserProfile, SitterData } from "@store/userStore";

export default async function MyProfilePage() {
  const { data: { user: authUser } } = await getServerUser();

  if (!authUser) return <MyProfileClient />;

  const db = createServiceClient();

  const [userResult, sitterResult] = await Promise.all([
    db
      .from("users")
      .select(
        "id, email, full_name, phone_number, address, display_area, latitude, longitude, birthdate, profile_image, is_verified, role, deleted_at",
      )
      .eq("id", authUser.id)
      .single(),
    getMySitterProfile(),
  ]);

  {...}

  return <MyProfileClient initialUser={initialUser} initialSitter={initialSitter} />;
}
```

MyProfileClient.tsx

```
//src/components/myprofile/MyProfileClient.tsx
"use client";

import { useState, useEffect, type ReactNode } from "react";
import { useRouter } from "next/navigation";

export default function MyProfileClient({
  initialUser,
  initialSitter,
}): {
  initialUser?: UserProfile | null;
  initialSitter?: SitterData | null;
} {
  const router = useRouter();
  const { user: storeUser, sitter: storeSitter, isLoading } = useUserStore();
  const user = storeUser ?? initialUser ?? null;
  const sitter = storeSitter ?? initialSitter ?? null;
  const isSitter = user?.role === "both" || user?.role === "admin";

  useEffect(() => {
    if (!isLoading && !user?.isVerified) {
      router.replace("/auth/verification");
    }
  }, [isLoading, user?.isVerified, router]);

  const [userType, setUserType] = useState<"owner" | "sitter">("owner");
  const [selectedMenu, setSelectedMenu] = useState("profile");
  const [locationData, setLocationData] = useState<LocationData | null>(null);
  const [showLocationModal, setShowLocationModal] = useState(false);
  const [prevUserLocation, setPrevUserLocation] = useState({
    address: user?.address,
    latitude: user?.latitude,
    longitude: user?.longitude,
    displayArea: user?.displayArea,
  });

  if (
    user?.address !== prevUserLocation.address ||
    user?.latitude !== prevUserLocation.latitude ||
    user?.longitude !== prevUserLocation.longitude ||
    user?.displayArea !== prevUserLocation.displayArea
  ) {
    setPrevUserLocation({
      address: user?.address,
      latitude: user?.latitude,
      longitude: user?.longitude,
      displayArea: user?.displayArea,
    });
    if (!user?.address || user?.latitude == null || user?.longitude == null) {
      setLocationData(null);
    } else {
      setLocationData({
        address: user.address,
        lat: user.latitude,
        lng: user.longitude,
        dong: user.displayArea || user.address,
      });
    }
  }
}
```

✓ RSC: 서버에서 미리 데이터 조회
RCC: 초기값 받아서 상태만 관리

04

프로젝트 진행 방식

팀 문화 · 협업 방식



01 데일리 스탠드업

매일 짧게 모여 어제 한 일 · 오늘 할 일 · 막힌 부분을 공유. 블록어는 그 자리에서 바로 담당자를 매칭

02 코드 리뷰 문화

PR 없이는 DEV에 머지하지 않는다는 원칙. 리뷰에서는 "왜 이렇게 짰는지"를 먼저 묻고, 스타일보다 로직·엡지케이스를 우선 확인

03 주간 회고

주 1회 KPT(KEEP·PROBLEM·TRY) 형식으로 짧게 회고하며 프로세스 자체를 계속 조정 할 일 · 막힌 부분을 공유. 그 자리에서 바로 담당자를 매칭

함께 이룬 성장

막히면 숨기지 않고 바로 알리는 것, 그리고 결정은 말로만 끝내지 않고 기록으로 남기는 것

기술적 성장

협업 성장

문제 해결 방식의 변화

커뮤니케이션

05

팀원 소개 및 소감

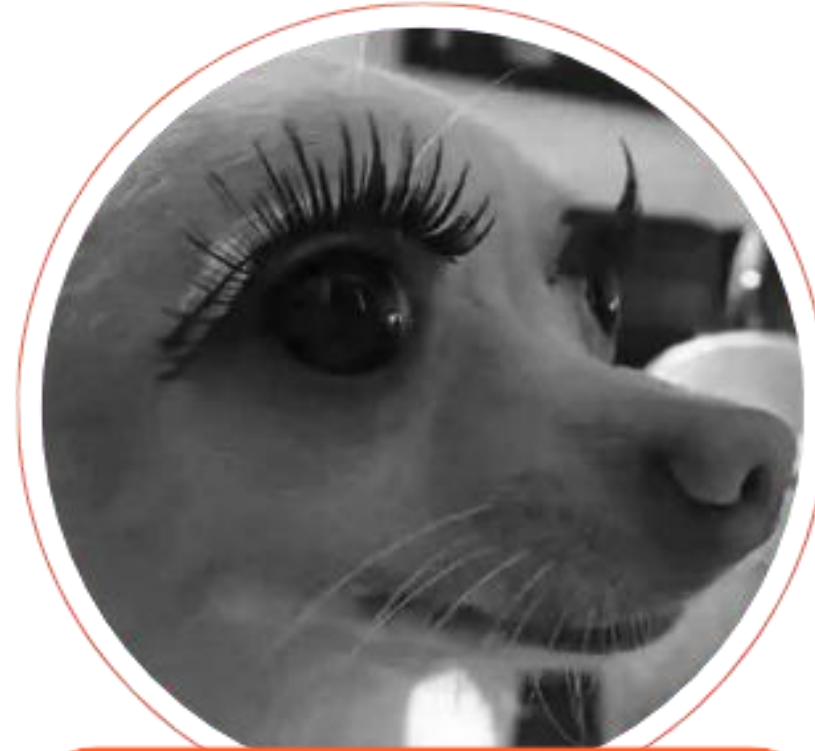
팀원 소개 및 소감



이규화

• GitHub @gyuhwa9922

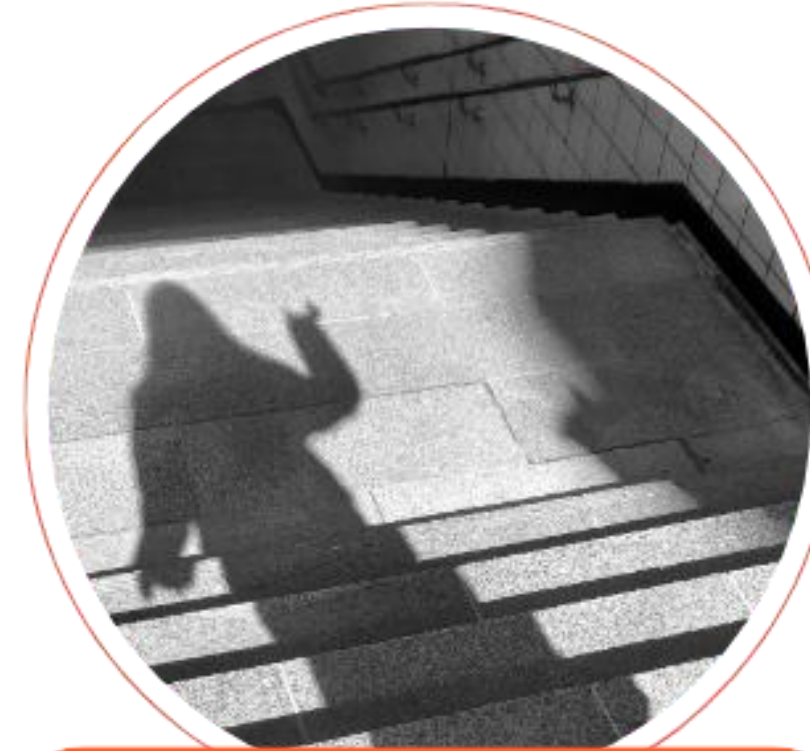
부족한 팀장이었지만
감사했습니다.



박규나

• GitHub @Gyu-me

기획 단계에서 미처 놓쳤던 부분들을 개발 과정에서 발견하게 되면서, 기획 단계의 중요성을 다시 한 번 느꼈습니다. 프로젝트를 마무리하며 부족했던 부분도 많이 느꼈지만, 그만큼 이후에는 추가적인 리팩토링을 통해 프로젝트를 개선하고 다시 회고해보고 싶다는 생각이 들었습니다. 프로젝트 기간 동안 팀장님과 팀원분들께 많은 것을 배울 수 있었습니다. 마지막 프로젝트까지 모두 정말 고생 많으셨습니다.



최영은

• GitHub @0sliverchoi321z

새로운 스택과 함께하니 바닐라 프로젝트와는 또 다른 느낌으로 임할 수 있어 좋았습니다. 강의 중 React나 Next.js를 배웠던 것보다도 심화 과정인 실전에 투입되었지만, 팀원들과의 논의와 멘토링, 코드 리뷰 피드백을 되짚어가며 부족한 부분을 채워나갈 수 있었습니다. 이런저런 시도가 많았지만 다시 한번 이번 작업물을 작업할 수 있는 기회가 있었으면 좋겠습니다. 마지막까지 다들 수고 많으셨고, 감사합니다.



이우현

• GitHub @sealheal

프로젝트를 하면서 React에 왜 Next.js를 사용하는지 라우팅과 SSR의 필요성에 대해서도 잘 이해하게 되었습니다. 모르는 게 많았지만, 팀원들 덕분에 하나의 서비스를 제공하는 데에는 정말 많은 부분에 대한 이해가 필요하다는 사실도 알게 되었습니다. 프로젝트를 진행하는 동안 고생해주셔서 감사합니다.

멋쟁이사자처럼 프론트엔드 부트캠프 17기

THANK YOU



와주개

믿고 맡기는
우리동네 펫시터 매칭

PRESENTER.

이규화

DATE.

2026.07.08

TEAM.

멍발자들